DEVSPHERE

AI-Powered Software Engineering & Code Intelligence Platform

Advanced BTech CSE Final-Year Placement Project
React + TypeScript • Java Spring Boot • Python FastAPI • PostgreSQL • Redis • Docker

Architecture, requirements, database design, APIs, starter code, AI integration, testing, deployment and GitHub workflow.

This is a detailed implementation blueprint with representative starter snippets, not a prebuilt fully tested repository. Implement and test each module, document your contribution and follow your college's academic-integrity rules.

1. Executive summary

DevSphere is a developer productivity and code intelligence platform. It connects to authorized source-code repositories, records pull-request and CI metadata, runs configurable static checks, summarizes changes and presents code-quality insights in one dashboard.

Why it is placement-oriented

• Demonstrates a multi-service architecture instead of a basic CRUD app.

• Combines Java backend engineering with a Python analysis service.

• Shows secure third-party API integration and webhook handling.

• Uses asynchronous jobs, caching, testing, CI/CD and deployment.

• Provides a clear end-to-end demo that can be explained in interviews.

MVP scope

Start with workspace management, repository connection, webhook ingestion, pull-request metadata, deterministic static checks and a dashboard. Add AI summaries only after the core workflow is reliable.

Boundary: DevSphere complements GitHub and static-analysis tools; it does not replace them. Never execute arbitrary repository code on your laptop or API server.

2. Users and features

Role	Features
Developer	Connect authorized repositories, view pull requests, run analysis, inspect findings and trends.
Team Lead	View repository health, configure quality thresholds and review high-priority findings.
Admin	Manage workspace membership, roles, integrations, audit logs and settings.

Core features

• Workspace and role-based access control.

• Repository connection using scoped provider permissions.

• Webhook ingestion for pull-request and CI events.

• Pull-request overview: author, changed files, checks and review state.

• Static checks for maintainability issues and configurable rules.

• AI-generated change summary and explanation of selected findings.

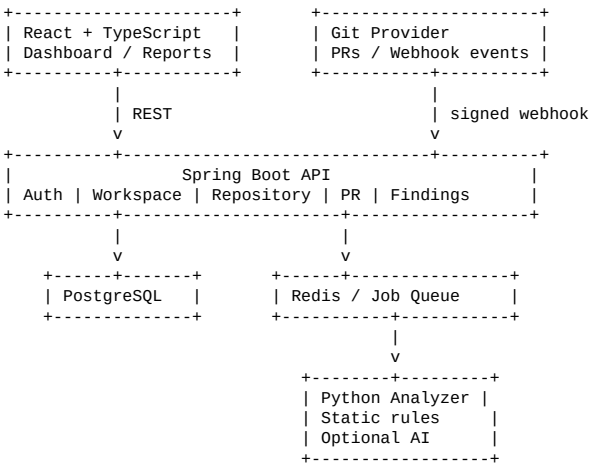
• Quality dashboard: findings by severity, trends and build success rate.

• Audit logs for integration and workspace actions.

3. Technology stack

Layer	Technology	Purpose
Frontend	React + TypeScript + Vite	Typed developer dashboard.
Backend	Java 21 + Spring Boot	REST APIs, validation, security and business rules.
Analysis service	Python + FastAPI	Isolated analysis jobs and optional AI.
Database	PostgreSQL	Workspace, repository and analysis data.
Queue/cache	Redis	Caching and lightweight asynchronous job coordination.
Integration	GitHub App / OAuth	Scoped repository access and webhook events.
Deployment	Docker Compose + cloud containers	Reproducible deployment.
Testing	JUnit, Mockito, Pytest, Playwright	Unit, integration and end-to-end tests.

4. Architecture



Event flow

• Developer connects an authorized repository.

• Provider sends a signed webhook when a pull request or workflow changes.

• Backend verifies signature and stores event idempotently.

• Backend schedules analysis; worker retrieves only permitted metadata or diff.

• Analyzer returns structured findings and optional AI summary.

• Backend stores results and exposes them to the dashboard.

Repository structure

```
devsphere/
  backend/src/main/java/com/devsphere/
    config/ controller/ dto/ entity/
    integration/ repository/ security/ service/
  analyzer/app/
    main.py rules.py
  frontend/src/
    api/ components/ pages/ hooks/
  infra/docker-compose.yml
  docs/architecture/ API.md
  README.md
```

5. Database design

Table	Important fields
users	id PK, name, email UNIQUE, password_hash
workspaces	id PK, name, owner_user_id FK
workspace_members	workspace_id FK, user_id FK, role; unique pair
integrations	id PK, workspace_id FK, provider, external_account_id, encrypted_token_ref
repositories	id PK, workspace_id FK, provider_repo_id, full_name, default_branch
pull_requests	id PK, repository_id FK, external_pr_id, title, author, state, head_sha
webhook_events	id PK, provider_event_id UNIQUE, event_type, received_at, processed_at, status
analysis_runs	id PK, repository_id FK, commit_sha, status, started_at, finished_at
findings	id PK, analysis_run_id FK, rule_id, severity, file_path, line_number, message
quality_gates	id PK, workspace_id FK, max_critical, max_high, enabled
audit_logs	id PK, workspace_id FK, actor_user_id FK, action, created_at

Data rules

• Unique provider event IDs prevent duplicate webhook effects.

• Every workspace-scoped query must verify membership and role.

• Store provider credentials encrypted or in a secret manager; never log raw tokens.

• Analysis runs are immutable after completion; create a new run for a new commit SHA.

• Index repository_id, analysis status, PR state and finding severity.

PostgreSQL starter schema

```
CREATE TABLE users (
 id BIGSERIAL PRIMARY KEY,
 name VARCHAR(120) NOT NULL,
 email VARCHAR(180) UNIQUE NOT NULL,
 password_hash VARCHAR(255) NOT NULL
);
CREATE TABLE workspaces (
 id BIGSERIAL PRIMARY KEY,
 name VARCHAR(120) NOT NULL,
 owner_user_id BIGINT NOT NULL REFERENCES users(id)
);
CREATE TABLE workspace_members (
 workspace_id BIGINT REFERENCES workspaces(id),
 user_id BIGINT REFERENCES users(id),
 role VARCHAR(20) NOT NULL CHECK
  (role IN ('OWNER','MAINTAINER','DEVELOPER','VIEWER')),
 PRIMARY KEY(workspace_id,user_id)
);
CREATE TABLE repositories (
 id BIGSERIAL PRIMARY KEY,
 workspace_id BIGINT NOT NULL REFERENCES workspaces(id),
 provider_repo_id VARCHAR(100) NOT NULL,
 full_name VARCHAR(250) NOT NULL,
 UNIQUE(workspace_id,provider_repo_id)
);
CREATE TABLE webhook_events (
 id BIGSERIAL PRIMARY KEY,
 provider_event_id VARCHAR(200) UNIQUE NOT NULL,
 event_type VARCHAR(100) NOT NULL,
 status VARCHAR(24) NOT NULL DEFAULT 'RECEIVED',
 received_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE TABLE analysis_runs (
 id BIGSERIAL PRIMARY KEY,
 repository_id BIGINT NOT NULL REFERENCES repositories(id),
 commit_sha VARCHAR(100) NOT NULL,
 status VARCHAR(20) NOT NULL,
 started_at TIMESTAMPTZ,
 finished_at TIMESTAMPTZ
);
CREATE TABLE findings (
 id BIGSERIAL PRIMARY KEY,
 analysis_run_id BIGINT NOT NULL REFERENCES analysis_runs(id),
 rule_id VARCHAR(80) NOT NULL,
 severity VARCHAR(16) NOT NULL,
 file_path VARCHAR(500) NOT NULL,
 line_number INT,
 message TEXT NOT NULL
);
CREATE INDEX idx_runs_repo_started
 ON analysis_runs(repository_id,started_at DESC);
```

6. REST API design

Endpoint	Access	Purpose
POST /api/auth/register	Public	Create account.
POST /api/auth/login	Public	Authenticate and issue token.
POST /api/workspaces	Authenticated	Create workspace.
GET /api/workspaces/{id}	Member	View workspace.
POST /api/workspaces/{id}/repositories	Maintainer+	Register repository.
GET /api/repositories/{id}/pull-requests	Member	List pull requests.
POST /api/repositories/{id}/analyses	Maintainer+	Queue analysis.
GET /api/analyses/{id}	Member	Get analysis and findings.
GET /api/workspaces/{id}/analytics	Member	Quality metrics.
POST /api/webhooks/github	Provider callback	Verify and accept event.

Example request

```
POST /api/repositories/17/analyses
Authorization: Bearer <access-token>
Content-Type: application/json

{
  "commitSha": "a1b2c3d4",
  "pullRequestNumber": 28
}
```

Example response

```
{
  "analysisId": 904,
  "status": "QUEUED",
  "commitSha": "a1b2c3d4"
}
```

Use 201 for created resources, 202 for queued work, 400 for invalid input, 401 for unauthenticated requests, 403 for forbidden access, 404 for missing resources and 409 for duplicate/conflicting events.

7. Backend starter code

Request DTO

```
package com.devsphere.dto;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Pattern;

public record CreateAnalysisRequest(
 @NotBlank
 @Pattern(regexp = "^[a-fA-F0-9]{7,64}$")
 String commitSha,
 Integer pullRequestNumber
) {}
```

Controller pattern

```
@RestController
@RequestMapping("/api/repositories")
public class AnalysisController {
 private final AnalysisService service;
 public AnalysisController(AnalysisService service) {
   this.service = service;
 }

 @PostMapping("/{repositoryId}/analyses")
 @PreAuthorize("hasAuthority('REPO_ANALYZE')")
 public ResponseEntity<AnalysisResponse> queue(
   @PathVariable Long repositoryId,
   @Valid @RequestBody CreateAnalysisRequest request,
   Authentication auth) {
   AnalysisResponse result =
     service.queue(repositoryId, request, auth.getName());
   return ResponseEntity.accepted().body(result);
 }
}
```

The service must verify workspace membership and repository permission, create a QUEUED analysis record and enqueue work transactionally. Configure the custom permission mapping in Spring Security.

Service responsibilities

• AnalysisService: authorize member, validate repository, create run and publish job.

• WebhookService: verify HMAC against raw request bytes, validate event type and deduplicate delivery ID.

• IntegrationService: call provider APIs with minimal scopes and handle rate limits.

• FindingService: validate analyzer output and persist findings transactionally.

• QualityGateService: evaluate configured thresholds and report pass/fail.

8. Python analyzer starter

Start with deterministic rules that are easy to explain and test. These educational checks are not a replacement for a mature static-analysis engine.

```
from fastapi import FastAPI
from pydantic import BaseModel, Field
from typing import List

app = FastAPI(title="DevSphere Analyzer")

class SourceFile(BaseModel):
    path: str = Field(max_length=500)
    content: str = Field(max_length=200000)
```

```
class AnalyzeRequest(BaseModel):
    files: List[SourceFile] = Field(max_length=100)

@app.post("/analyze")
def analyze(payload: AnalyzeRequest):
    findings = []
    for file in payload.files:
        for number, line in enumerate(file.content.splitlines(), 1):
            if "TODO" in line:
                findings.append({
                    "ruleId": "TODO_MARKER",
                    "severity": "LOW",
                    "filePath": file.path,
                    "lineNumber": number,
                    "message": "Unresolved TODO marker."
                })
            if len(line) > 120:
                findings.append({
                    "ruleId": "LONG_LINE",
                    "severity": "INFO",
                    "filePath": file.path,
                    "lineNumber": number,
                    "message": "Line exceeds 120 characters."
                })
    return {"findings": findings, "engine": "devsphere-rules-v1"}
```

Run with: python -m venv .venv; activate it; pip install fastapi uvicorn pydantic; uvicorn app.main:app --reload --port 8001.

AI extension

• Summarize a pull-request diff and explain selected findings.

• Send only minimum permitted diff context; exclude secrets and unrelated files.

• Treat repository content as untrusted input, never as model instructions.

• Validate model output against a strict schema and label it AI-generated.

• Evaluate factual error rate and usefulness with human-reviewed examples.

9. Frontend screens

Screen	Components
Login / Signup	Authentication forms, validation and error states.
Workspace home	Repository list, recent analysis runs and quality overview.
Repository detail	PR list, branch/commit info, latest checks and trends.
PR analysis	Changed files, severity-grouped findings, AI summary and quality gate.
Findings explorer	Filter by rule, severity, file path and analysis run.
Workspace settings	Members, roles, integration and quality thresholds.

React API client

```
import axios from "axios";
const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL || "http://localhost:8080/api"
});
api.interceptors.request.use((config) => {
  const token = sessionStorage.getItem("accessToken");
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});
export default api;
```

Findings list component

```
function FindingsList({ findings }) {
  if (!findings?.length) return <p>No findings for this run.</p>;
  return (
    <section aria-label="Code findings">
      {findings.map((item) => (
        <article key={item.id}>
          <strong>{item.severity}</strong>
          <span> {item.ruleId} </span>
          <p>{item.message}</p>
          <small>{item.filePath}:{item.lineNumber ?? "-"}</small>
        </article>
      ))}
    </section>
  );
}
```

10. macOS local setup

```
java -version
node -v
npm -v
python3 --version
docker --version

# Backend
cd backend
./mvnw spring-boot:run

# Analyzer (project root)
cd analyzer
python3 -m venv .venv
source .venv/bin/activate
pip install fastapi uvicorn pydantic
uvicorn app.main:app --reload --port 8001

# Frontend
cd frontend
npm install
npm run dev
```

Run PostgreSQL and Redis locally through Docker Compose. Set credentials and signing keys through environment variables or local untracked configuration. Never commit secrets.

11. Testing strategy

Layer	Tests
Backend unit	Workspace permission checks, quality-gate thresholds, webhook deduplication.
Integration	Database constraints, API validation, transaction rollback and queries.
Analyzer	Rule fixtures, clean files, malformed payloads and input limits.
Frontend	Empty/loading/error states, filters, navigation and access denied.
End-to-end	Demo repository event → analysis → findings → quality result.
Security	Invalid webhook signature, replayed event, cross-workspace access, token expiry.

Definition of done

• Every endpoint has validation and authorization tests.

• Webhook processing is idempotent and signature-verified.

• Analysis states include QUEUED, RUNNING, SUCCEEDED and FAILED.

• Logs use correlation IDs and never expose secrets.

• README setup works from a clean clone.

12. Security and reliability

• Use BCrypt or Argon2 for passwords and short-lived access tokens.

• Use least-privilege repository integration scopes.

• Verify webhook signatures against raw request body and reject invalid signatures.

• Treat source code, filenames, PR descriptions and comments as untrusted content.

• Never execute arbitrary repository code on the API server.

• If build execution is added, use disposable isolated sandboxes with strict resource and network limits.

• Encrypt provider credentials at rest or store references in a secret manager.

• Add rate limiting, request-size limits, audit logging and dependency scanning.

Academic MVP safety: Analyze text or provider-supplied diff metadata without running untrusted build scripts.

13. Docker and deployment

```
services:
  postgres:
```

```
    image: postgres:16
    environment:
      POSTGRES_DB: devsphere
      POSTGRES_USER: devsphere_user
      POSTGRES_PASSWORD: local_dev_password
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
  redis:
    image: redis:7
    ports:
      - "6379:6379"
volumes:
  pgdata:
```

• Run backend and analyzer tests; build container images.

• Provision managed PostgreSQL and Redis.

• Set production secrets in the hosting provider's secret manager.

• Deploy backend and analyzer as separate services.

• Deploy frontend static assets with production API URL.

• Configure HTTPS, CORS, webhook endpoint and provider callback URL.

• Enable logs, metrics, health checks, backups and alerts.

• Run smoke tests with a demo repository.

14. GitHub push workflow

```
# From devsphere project root
git init
git add .
git commit -m "Initial DevSphere project"
git branch -M main
git remote add origin https://github.com/YOUR-USERNAME/devsphere.git
git push -u origin main
```

.gitignore essentials

```
target/
node_modules/
dist/
.venv/
__pycache__/
*.pyc
.env
.env.*
!.env.example
application-local.properties
.DS_Store
```

Create a GitHub description and topics such as java, spring-boot, react, python, developer-tools, code-quality and docker. Include diagrams, screenshots, a demo video and a Postman collection.

15. README outline

```
# DevSphere
AI-assisted software engineering and code intelligence platform.

## Features
- Workspace and repository dashboard
- Pull request and CI event ingestion
- Static code-quality checks
- Analysis findings and quality gates
- Optional AI-generated change summaries

## Stack
React, TypeScript, Java 21, Spring Boot, Python FastAPI,
PostgreSQL, Redis and Docker.

## Local development
See docs/SETUP.md for prerequisites, environment variables,
database setup, service startup and test commands.

## Security
Never commit credentials, access tokens or private source code.
Use demo repositories and least-privilege integration scopes.
```

16. Eight-week build plan

Week	Deliverable
1	Requirements, wireframes, architecture, ER diagram and repository.
2	Spring Boot setup, users, workspaces, roles and authentication.
3	Repository records and provider integration in demo environment.
4	Webhook signature verification, event storage and idempotency.
5	Python static checks, analysis jobs and findings persistence.
6	React dashboard, PR analysis view and quality gate.
7	AI summary extension, tests, security review and bug fixes.
8	Docker deployment, screenshots, report and demo rehearsal.

17. Placement interview questions

Question	What to explain
Why split Java and Python?	Java for core domain/security; Python for analysis ecosystem.
What is a webhook?	An event callback notifying an app of a provider-side change.
How verify webhook?	Validate HMAC signature over the raw request body.
What is idempotency?	Repeated delivery of the same event has no duplicate effect.
Why use a queue?	Long-running analysis is decoupled from HTTP latency.
How protect workspace data?	Authorize each operation against membership and role.
What is a quality gate?	Configured rules that decide whether an analysis passes.
How avoid unsafe code execution?	Do not execute arbitrary code; if needed, use isolated ephemeral sandboxes.
How measure analyzer quality?	Use labelled fixtures, precision/recall and false-positive review.

18. Final submission checklist

- Backend, analyzer and frontend run from documented setup.
- Database migrations and synthetic sample data are included.
- Tests cover permissions, webhook security, idempotency and analyzer rules.
- Dashboard screenshots and architecture diagrams are included.
- README distinguishes implemented features from planned features.
- No credentials, private tokens or confidential repository content are committed.
- Demo is repeatable from a clean clone.
- You can explain architecture, data model, security and trade-offs.

Interview demo: Show a pull-request event arriving, verify it in the backend, run the analyzer, inspect findings and explain the quality gate. Discuss a real bug you fixed and how your tests prevent regression.

End of DevSphere project guide.